
Documento de Contratos

Low Accuracy Kinda Intelligent Tracking Unit (LAKITU)



Integrantes:

Agustín Cayrus

Sofía Dutra

Ignacio Moreno

Santiago Peña

Fecha:

06 de Abril de 2026

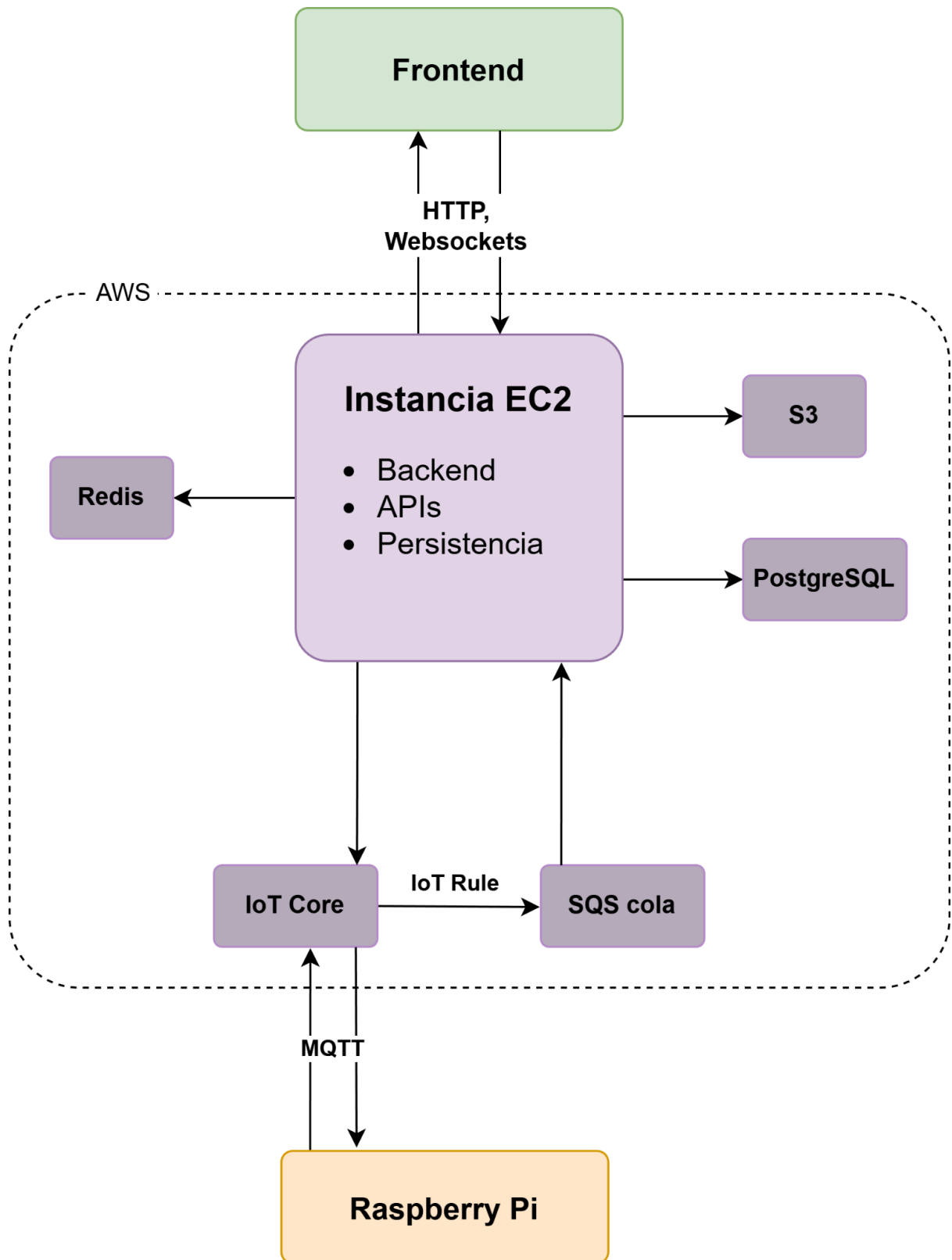
Índice

Índice.....	2
Glosario.....	3
Mapa del sistema.....	4
Contratos de datos.....	5
Raspberry Pi → IoT Core → SQS → Cloud Receptor.....	5
Notas.....	5
Payload.....	5
Websocket API → Frontend.....	6
Notas.....	6
Payload.....	6
Frontend → Websocket API → (...) → Raspberry Pi.....	6
Notas.....	6
Payload.....	6
Respuesta.....	6
REST API: Endpoint /api/signup.....	7
Notas.....	7
Response 200 OK.....	7
Response 409 400 500.....	7
REST API: Endpoint /api/login.....	7
Notas.....	7
Response 200 OK.....	8
Response 401 400 500.....	8
REST API: Endpoint /api/events.....	8
Notas.....	8
Response 200 OK.....	8
Response 401 400 500.....	9
REST API: Endpoint /api/stats/(?)......	9
Notas.....	9
Response 200 OK.....	9
Esquemas.....	10
PostgreSQL.....	10
Diagrama entidad-relación.....	10
Esquema UML.....	10
Redis.....	11
Forma de los datos.....	11
Escritura en Redis.....	11
Lectura en Redis.....	11

Glosario

Término	Definición
Lugar / Spot	Un lugar físico único del estacionamiento, identificado por un ID único.
Estado de un lugar / Spot status	0 o 1, para libre y ocupado respectivamente.
Estado (del estacionamiento) / State	Conjunto de los estados de todos los lugares del estacionamiento.
Actualización de estado / State update	Mensaje enviado por el Pi, conteniendo el estado del estacionamiento y otros datos relevantes, cuando algún lugar puntual cambia su estado.
Confianza / Confidence	Un float en el intervalo [0. 0, 1. 0] que representa la certeza que tiene el sistema sobre el estado de un lugar determinado.
Evento	Un registro almacenado de una actualización de estado recibida.

Mapa del sistema



Contratos de datos

Raspberry Pi → IoT Core → SQS → Cloud Receptor

Notas

- Objetivo: Comunicar una actualización de estado al backend.
- Protocolo: MQTT (Pi → IoT Core), IoT Core Rule (IoT Core → SQS), HTTPS
- Topic: parking/state_update
- Restricción de tamaño: IoT Core impone un límite de transmisión de 128Kb.

Payload

```
{
  "device_id": "string",
  "parking_id": "string",
  "parking_name": "string",
  "timestamp": "ISO 8601 UTC string",
  "seq": "integer",
  "snapshot": "byte[]",
  "total_spots": "integer",
  "free_spots": "integer",
  "spots": [
    {
      "spot_id": "string",
      "status": "0 | 1",
      "confidence": "float 0.0-1.0",
      "last_changed": "ISO 8601 UTC string"
    }
  ]
}
```

- **"timestamp"** es el momento en el que el Raspberry Pi captura la imagen, no el momento en el que la envía.
- Si un lugar cambia su estado en esta transmisión, el **"last_changed"** de ese lugar debe ser igual a **"timestamp"**.
- **"status"** es 0 si el lugar está libre y 1 si está ocupado.
- **"seq"** es un número de secuencia incremental.

Websocket API → Frontend

Notas

- Objetivo: Comunicar actualizaciones de estado a los usuarios actualmente conectados.
- Protocolo: Websocket (HTTPS para establecer la conexión, luego TCP).
- Endpoint: wss://nuestra-url/ws

Payload

```
{
  "type": "STATE_UPDATE | INITIAL_STATE",
  "parking_id": "string",
  "parking_name": "string",
  "timestamp": "ISO 8601 UTC string",
  "spots": [
    { "spot_id": "string", "status": "0 | 1" }
  ]
}
```

- **"type"** solo será **"INITIAL_STATE"** en la comunicación HTTPS inicial que establece la conexión

Frontend → Websocket API → (...) → Raspberry Pi

Notas

- Objetivo: Comandar la captura de una foto actual.
- Protocolo: Websocket (HTTPS para establecer la conexión, luego TCP), No se (Cloud Receptor → IoT Core), MQTT (IoT Core → Raspberry Pi).

Payload

```
{
  "action": "TAKE_PHOTO",
  "requestId": "integer"
}
```

Respuesta

```
{
  "requestId": "integer",
  "data": "byte[]",
}
```

```
"status": "ok | string",
}
```

- "status" será el mensaje de error en caso de que algo falle.

REST API: Endpoint /api/signup

Notas

- Objetivo: Registrarse.
- POST
- Query params:
 - email
 - password
 - name
- Posibles respuestas:
 - 200 OK
 - 409 Conflict
 - 400 Bad request - Query params faltantes o en mal formato
 - 500 Internal server error

Response 200 OK

```
{
  "user": {
    "id": "integer",
    "email": "string"
  },
  "token": "string"
}
```

Response 409 | 400 | 500

```
{
  "error": "string"
}
```

REST API: Endpoint /api/login

Notas

- Objetivo: Iniciar sesión.
- Query params:

- email
 - password
- Posibles respuestas:
 - 200 OK
 - 401 Unauthorized
 - 400 Bad request - Query params faltantes o en mal formato
 - 500 Internal server error

Response 200 OK

```
{
  "accessToken": "jwt_access_token",
  "user": {
    "id": "integer",
    "email": "string"
  }
}
```

Response 401 | 400 | 500

```
{
  "error": "string"
}
```

REST API: Endpoint /api/events

Notas

- Objetivo: Pedir eventos pasados.
- Query params:
 - **from** - datetime, obligatorio
 - **to** - datetime, obligatorio
 - **limit** - integer, opcional, default 20
 - **page** - integer, opcional, default 1
- Posibles respuestas
 - 200 OK
 - 400 Bad Request - Query params faltantes o en mal formato
 - 500 Internal server error

Response 200 OK

```
{
```



```
"total_events": "integer",
"events": [
  {
    "id": "integer",
    "pi_timestamp": "ISO 8601 UTC string",
    "free_spots": "integer",
    "total_spots": "integer",
    "image_url": "string",
    "spots": [
      { "spot_id": "string", "status": "0 | 1" }
    ]
  }
]
```

- **"total_events"** es la cantidad total de eventos registrados entre **from** y **to**.

Response 401 | 400 | 500

```
{
  "error": "string"
}
```

REST API: Endpoint /api/stats/(?)

Notas

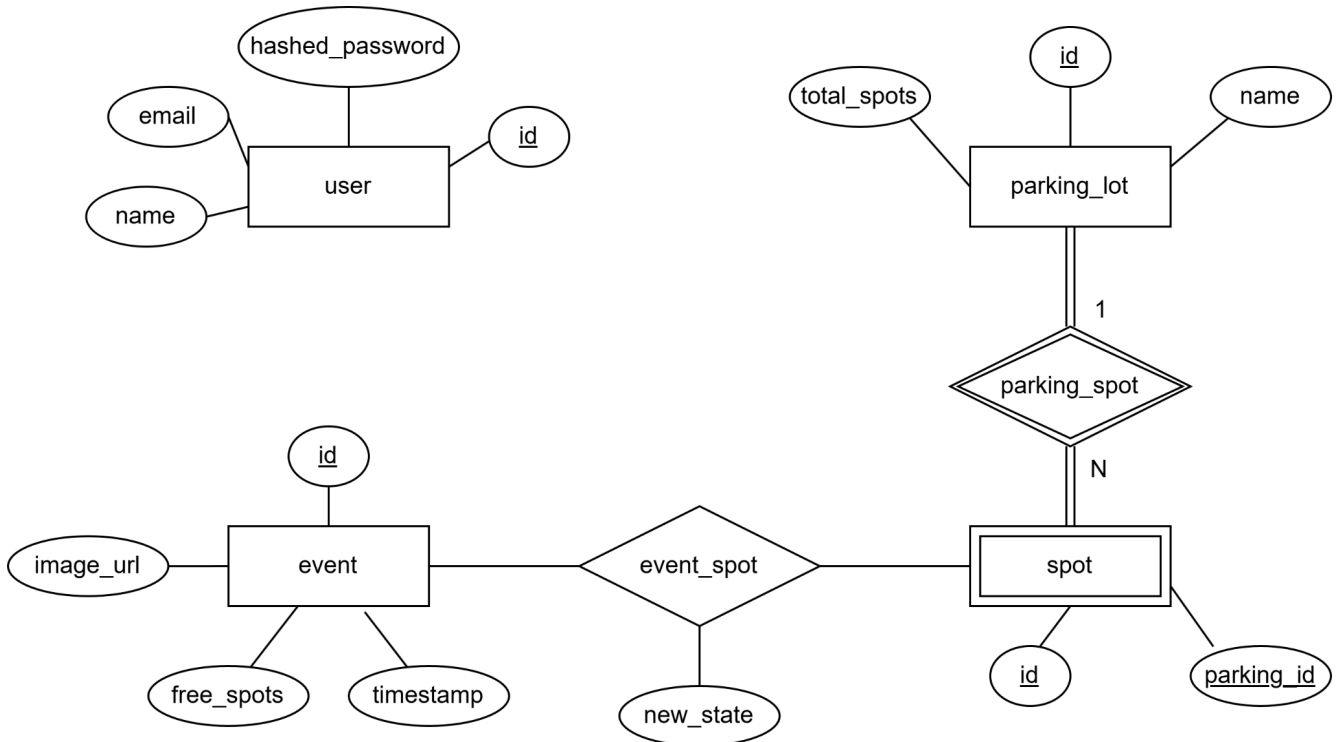
- Objetivo: Pedir estadísticas.
-

Response 200 OK

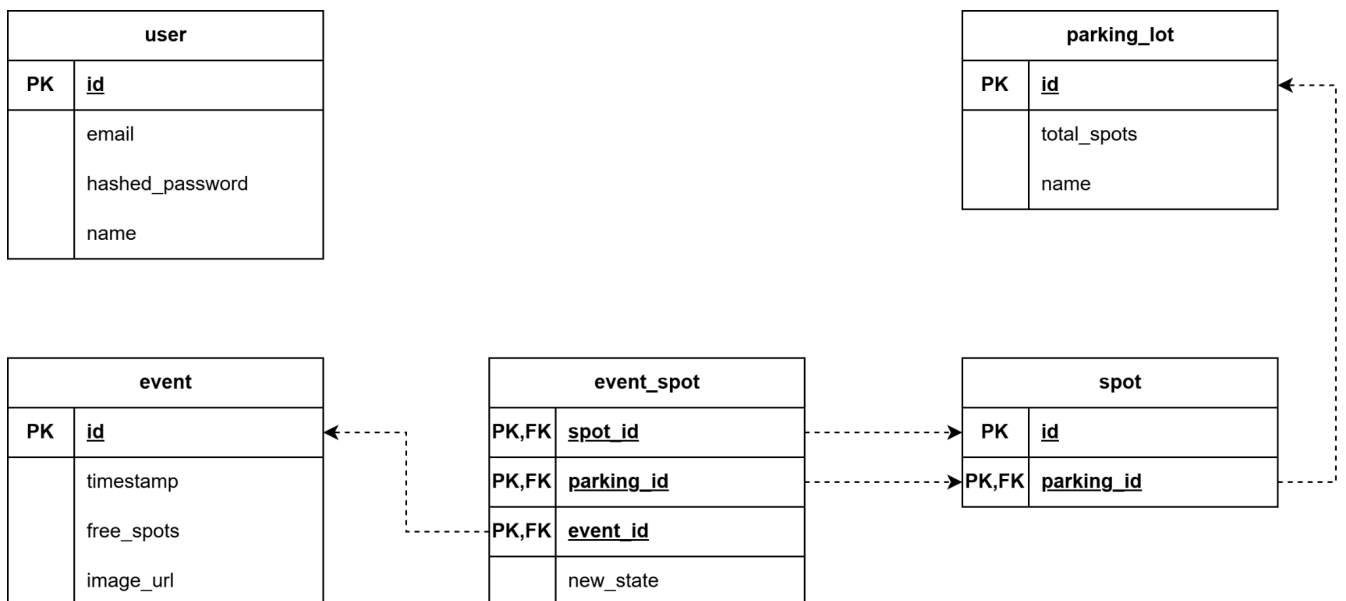
Esquemas

PostgreSQL

Diagrama entidad-relación



Esquema UML



Redis

Redis puede almacenar hashes como par key:value, strings, y conjuntos ordenados o no ordenados. Nosotros vamos a usar un string para almacenar un JSON serializado en `parking_lot:{parking_id}:state`.

Forma de los datos

```
{
  "parking_id": "string",
  "parking_name": "string",
  "timestamp": "ISO 8601 UTC string",
  "spots": [
    { "spot_id": "string", "status": "0 | 1" }
  ]
}
```

Escritura en Redis

```
import json, redis

r = redis.Redis(host=REDIS_HOST)

state = {
    "parking_id": event.parking_id,
    "parking_name": event.parking_name,
    "timestamp": event.timestamp,
    "spots": {
        s.spot_id: s.status
        for s in event.spots
    }
}

r.set(f"parking_lot:{event.parking_id}:state", json.dumps(state))
```

Lectura en Redis

```
raw = r.get(f"parking_lot:{parking_id}:state")

if raw is None:
    state = query_latest_event_from_postgres(parking_id)
    r.set(f"parking_lot:{parking_id}:state", json.dumps(state))
else:
    state = json.loads(raw)
send_to_client({"type": "INITIAL_STATE", **state})
```